Campus City Emergency Supply Distribution

Integer Linear Programming Approach using PuLP

24MAT382 – Computational Optimization & Applications

Department of Mathematics
Saintgits College of Engineering (Autonomous)

Name: Akhila Sunesh
Roll No: 16

March 9, 2026

Abstract

Efficient emergency logistics is a critical component of disaster preparedness and operational resilience in institutional campuses. This project presents an optimization framework for designing an effective emergency supply distribution network across facilities in a campus city environment.

The objective of the study is to determine optimal warehouse locations and shipment quantities that minimize the total annual cost while satisfying facility demand, respecting warehouse capacity constraints, maintaining redundancy, and remaining within a specified budget limit.

The problem is formulated as a Mixed Integer Linear Programming (MILP) model and implemented in Python using the PuLP optimization library. Three potential warehouse locations and six essential campus facilities are considered in the model. A redundancy constraint ensures that exactly two warehouses are selected to improve reliability and operational robustness.

The optimization results demonstrate that mathematical programming provides a systematic and cost-effective strategy for emergency supply distribution planning.

Project Repository

The complete source code, datasets, and implementation files for this project are available in the GitHub repository below:

GitHub Repository: CampusCity-Supply-Optimization-MP1

The repository contains the Python implementation, dataset files used in the optimization model, and visualization outputs generated as part of this study.

Contents

1 Introduction **6**

2 Problem Statement **6**

3 Data Description **6**

3.1 Facilities Data 6

3.2 Warehouse Data 7

4 Mathematical Formulation **7**

4.1 Decision Variables 7

4.2 Objective Function 8

5 Constraints **8**

5.1 Demand Satisfaction 8

5.2 Warehouse Capacity 8

5.3 Warehouse Selection 8

5.4 Budget Constraint 8

6 Methodology **8**

7 Libraries Used **8**

8 Results **9**

9 Visualization of Distribution Network **10**

10 Discussion **11**

11 Limitations **11**

12 Conclusion **12**

A Python Implementation Code **12**

List of Figures

1 Optimized Emergency Supply Distribution Network 10
2 Interactive Geographic Visualization 11

1 Introduction

Large institutional campuses operate similarly to small urban environments, containing residential buildings, academic centers, healthcare facilities, and essential service infrastructure. In emergency situations such as natural disasters, pandemics, or infrastructure failures, the timely distribution of critical supplies becomes essential for maintaining operational continuity and safeguarding human welfare.

Traditional supply allocation approaches often rely on heuristic decision-making or manual planning, which may lead to inefficient transportation routes, underutilized storage capacity, and increased operational costs. Furthermore, a lack of redundancy in supply systems can increase vulnerability during infrastructure disruptions.

This project applies mathematical optimization techniques to design a cost-efficient emergency supply distribution system for a campus environment. The study focuses on determining optimal warehouse selection and shipment allocation strategies that minimize total operational costs while meeting demand requirements and maintaining system reliability.

2 Problem Statement

The primary objective of the problem is to determine the optimal configuration of warehouse locations and supply flows that minimizes the total annual distribution cost. The system must satisfy the demand requirements of all facilities across the campus while respecting the capacity limitations of the warehouses.

Additionally, the model enforces the selection of exactly two warehouses to ensure redundancy in emergency situations. The solution must also remain within a predefined annual budget of $1,500,000.

3 Data Description

3.1 Facilities Data

The campus includes six key facilities that require emergency supplies.

Facility ID	Type	Daily Demand (Units)
MED_CENTER	Hospital	80
ENG_BUILDING	Academic	30
SCIENCE_HALL	Academic	35
DORM_A	Residential	55
DORM_B	Residential	45
LIBRARY	Academic	25

The combined daily demand across all facilities is 270 units.

$$D_f^{annual} = D_f^{daily} \times 365$$

Total annual demand becomes **98,550 units**.

3.2 Warehouse Data

Warehouse	Daily Capacity	Construction Cost	Operational Cost/Day
WH_NORTH	400	$300,000	$800
WH_SOUTH	350	$280,000	$700
WH_EAST	450	$320,000	$900

Construction costs are amortized over **10 years**, while operational costs are calculated annually.

4 Mathematical Formulation

The emergency distribution problem is modeled as a Mixed Integer Linear Programming (MILP) problem.

4.1 Decision Variables

Binary variable:

$$y_w = \begin{cases} 1 & \text{if warehouse } w \text{ is opened} \\ 0 & \text{otherwise} \end{cases}$$

Shipment variable:

$$x_{wf}$$

represents the number of units shipped annually from warehouse w to facility f.

4.2 Objective Function

$$\min Z = \sum_{w \in W} \sum_{f \in F} c_{wf} x_{wf} + \sum_{w \in W} F_w y_w$$

5 Constraints

5.1 Demand Satisfaction

$$\sum_{w \in W} x_{wf} = D_f^{annual} \quad \forall f$$

5.2 Warehouse Capacity

$$\sum_{f \in F} x_{wf} \leq Cap_w^{annual} y_w$$

5.3 Warehouse Selection

$$\sum_{w \in W} y_w = 2$$

5.4 Budget Constraint

$$\sum_{w} F_w y_w + \sum_{w} \sum_{f} c_{wf} x_{wf} \leq 1{,}500{,}000$$

6 Methodology

The model is implemented using Python and the PuLP optimization library. Data is imported from structured CSV files containing facility demand, warehouse capacity, and transportation costs.

The steps include:

1. Data preprocessing
2. MILP model formulation
3. Optimization using PuLP solver
4. Visualization of results

7 Libraries Used

The implementation of the optimization model and the accompanying visualizations were carried out using several Python libraries. These libraries play an important role in data

processing, mathematical modeling, and graphical representation of results. By combining these tools, the project efficiently handles data preparation, optimization computation, and visualization of the distribution network.

Pandas was used for data preprocessing and management. It allows structured datasets such as facility demand, warehouse capacity, and transportation costs to be loaded from CSV files and manipulated efficiently. Operations such as filtering, indexing, and data transformation were performed using Pandas to prepare the data for optimization.

PuLP serves as the core optimization library used in this project. It enables the formulation of Linear Programming and Mixed Integer Linear Programming (MILP) models in Python. Using PuLP, the decision variables, objective function, and system constraints were defined, and the solver was used to compute the optimal warehouse selection and shipment distribution.

Matplotlib was used to generate static visualizations of the optimized distribution network. It helps represent warehouses, facilities, and shipment routes on a coordinate-based map. These visualizations make it easier to interpret the optimization results and understand the relationships between supply points and demand locations.

Folium was used to create an interactive geographic map of the distribution system. Unlike static plots, Folium allows users to zoom, pan, and interact with markers representing warehouses and facilities. Shipment routes can also be visualized dynamically, providing a more intuitive understanding of the optimized supply network.

- **Pandas** – Data preprocessing and structured dataset handling
- **PuLP** – Mathematical optimization modeling and solver interface
- **Matplotlib** – Static visualization of the distribution network
- **Folium** – Interactive geographic visualization

8 Results

The solver successfully produced an optimal solution satisfying all constraints. Two warehouses were selected to supply all campus facilities while minimizing total annual cost and staying within the budget.

9 Visualization of Distribution Network

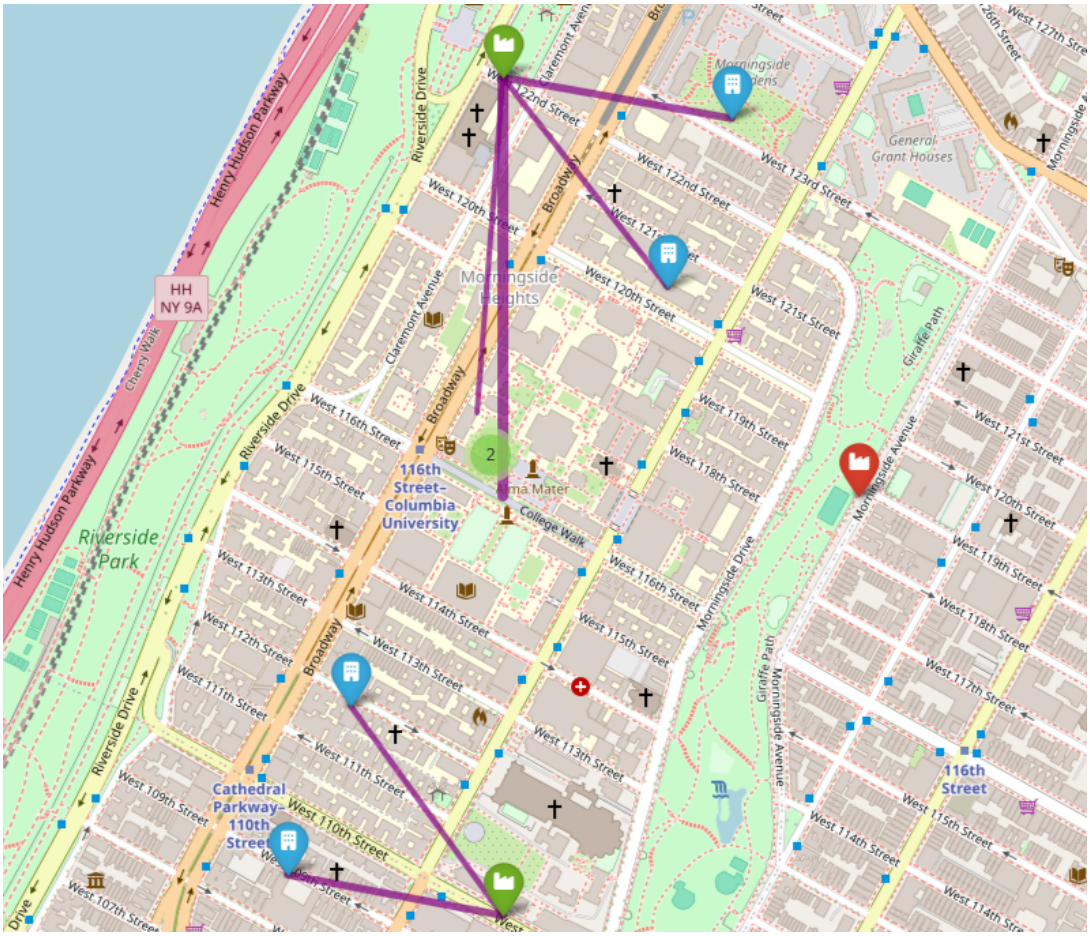

Figure 1: Optimized Emergency Supply Distribution Network

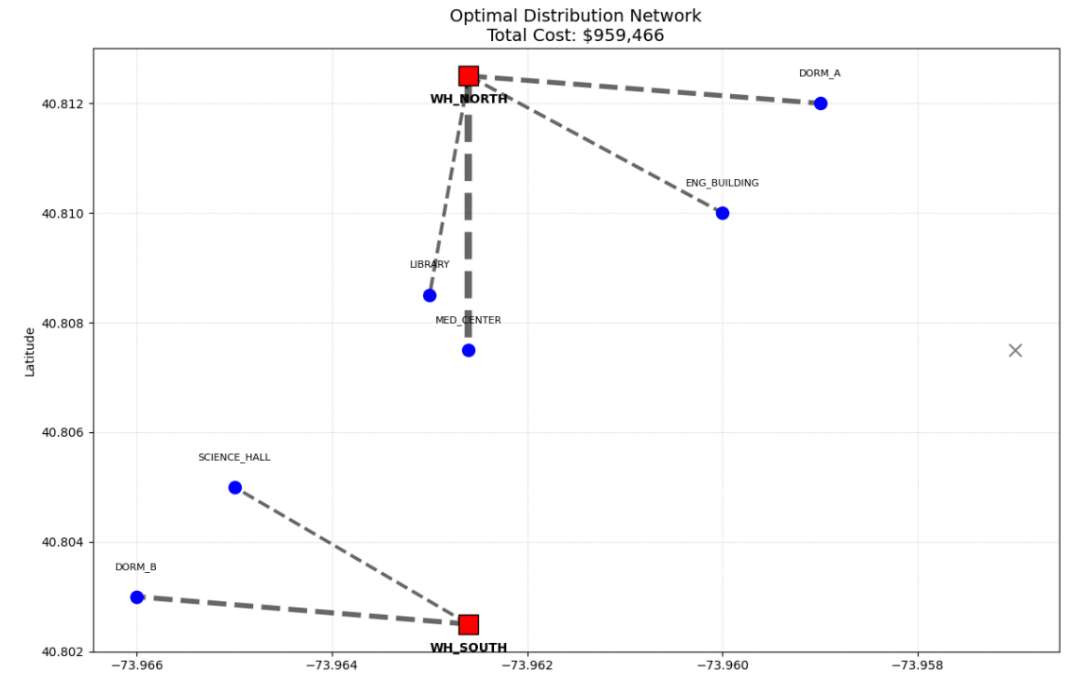

Figure 2: Interactive Geographic Visualization

10 Discussion

The results highlight the importance of balancing infrastructure costs and transportation costs when designing logistics networks. Redundancy through two warehouse selections improves reliability and resilience.

11 Limitations

While the optimization model provides an effective framework for emergency supply distribution, several simplifying assumptions were made in order to keep the model tractable and suitable for academic implementation. These assumptions introduce certain limitations that may affect the applicability of the model in real-world emergency logistics scenarios.

One of the primary limitations of the model is the assumption of deterministic demand. In the current formulation, the daily demand values for each facility are considered fixed and known in advance. However, in real emergency situations, demand can fluctuate significantly depending on the severity of the situation, population movement, or unforeseen circumstances. Incorporating stochastic or uncertain demand models would make the system more realistic and robust.

Another limitation is the absence of transportation delays or disruptions. The model assumes that shipments can be transported instantly and reliably from warehouses to facilities without any interruptions. In reality, road conditions, weather events, traffic

congestion, and infrastructure failures can significantly affect delivery times. Accounting for such uncertainties would require more complex models involving time-dependent routing or simulation techniques.

The model also uses a single-period planning horizon. All decisions are optimized based on aggregated annual demand rather than considering changes over time. In practical logistics planning, demand patterns, resource availability, and operational costs can vary across days, weeks, or months. A multi-period optimization framework would allow planners to adjust supply strategies dynamically over time.

The main limitations of the current model can therefore be summarized as follows:

- **Deterministic demand assumption** – Demand values are fixed and do not account for uncertainty or variability during emergencies.
- **No transportation delays** – The model assumes ideal transportation conditions without disruptions or travel time constraints.
- **Single-period planning** – The optimization is performed for a single aggregated time period rather than multiple time horizons.

12 Conclusion

This project demonstrates the application of Integer Linear Programming in emergency logistics planning. The optimization framework provides a scalable and cost-efficient method for supply distribution across institutional campuses.

Future work can include stochastic demand models, multi-period planning, and real-time logistics optimization.

References

1. 24MAT382 Course Notes
2. PuLP Documentation
3. Hillier, F. S., & Lieberman, G. J., *Introduction to Operations Research*

A Python Implementation Code

```
# ============================================================
# CAMPUS CITY EMERGENCY DISTRIBUTION OPTIMIZATION
# Complete Model + Visualization
# ============================================================
```

```python
import pandas as pd
import pulp
import matplotlib.pyplot as plt

# ============================================================
# 1. LOAD DATA
# ============================================================

print("Loading data...")

demands_df = pd.read_csv("demands.csv")
warehouses_df = pd.read_csv("warehouses.csv")
transport_df = pd.read_csv("transportation_costs.csv")
facilities_df = pd.read_csv("facilities.csv")

# Clean column names
for df in [demands_df, warehouses_df, transport_df, facilities_df
    ]:
    df.columns = df.columns.str.strip().str.lower()

# ============================================================
# 2. PARAMETERS
# ============================================================

TARGET_FACILITIES = [
    'MED_CENTER', 'ENG_BUILDING', 'SCIENCE_HALL',
    'DORM_A', 'DORM_B', 'LIBRARY'
]

TARGET_WAREHOUSES = ['WH_NORTH', 'WH_SOUTH', 'WH_EAST']

DAYS_PER_YEAR = 365
AMORTIZATION_YEARS = 10
BUDGET_LIMIT = 1500000

# ============================================================
# 3. DATA PREPARATION
# ============================================================

# ---- Demand (Annualized)
```

```
demands_subset = demands_df[
    demands_df['facility_id'].isin(TARGET_FACILITIES)
].set_index('facility_id')

annual_demand = (
    demands_subset['daily_demand'] * DAYS_PER_YEAR
).to_dict()

# ---- Warehouse Capacity (Annualized)
warehouses_subset = warehouses_df[
    warehouses_df['warehouse_id'].isin(TARGET_WAREHOUSES)
].set_index('warehouse_id')

annual_capacity = (
    warehouses_subset['capacity'] * DAYS_PER_YEAR
).to_dict()

# ---- Annual Fixed Cost
annual_fixed_cost = {}
for wh_id, row in warehouses_subset.iterrows():
    annual_fixed_cost[wh_id] = (
        (row['construction_cost'] / AMORTIZATION_YEARS)
        + (row['operational_cost'] * DAYS_PER_YEAR)
    )

# ---- Transport Cost Dictionary
transport_costs = {}
for _, row in transport_df.iterrows():
    if row['from_warehouse'] in TARGET_WAREHOUSES and \
       row['to_facility'] in TARGET_FACILITIES:
        transport_costs[(row['from_warehouse'],
                         row['to_facility'])] = row['
                            cost_per_unit']

# ============================================================
# 4. OPTIMIZATION MODEL
# ============================================================

print("Solving optimization model...")

prob = pulp.LpProblem("Campus_City_Distribution",
```

```
                        pulp.LpMinimize)

# Decision Variables
y = pulp.LpVariable.dicts("Open",
                          TARGET_WAREHOUSES,
                          cat='Binary')

x = pulp.LpVariable.dicts("Ship",
                          (TARGET_WAREHOUSES,
                           TARGET_FACILITIES),
                          lowBound=0,
                          cat='Continuous')

# ---- Objective Function
prob += (
    pulp.lpSum(annual_fixed_cost[j] * y[j]
               for j in TARGET_WAREHOUSES)
    +
    pulp.lpSum(
        transport_costs[(j, i)] * x[j][i]
        for j in TARGET_WAREHOUSES
        for i in TARGET_FACILITIES
    )
)

# ---- Constraints

# Demand Satisfaction
for i in TARGET_FACILITIES:
    prob += (
        pulp.lpSum(x[j][i]
                   for j in TARGET_WAREHOUSES)
        == annual_demand[i]
    )

# Capacity Constraints
for j in TARGET_WAREHOUSES:
    prob += (
        pulp.lpSum(x[j][i]
                   for i in TARGET_FACILITIES)
        <= annual_capacity[j] * y[j]
```

```
    )

# Select Exactly 2 Warehouses
prob += pulp.lpSum(y[j] for j in TARGET_WAREHOUSES) == 2

# Budget Constraint
prob += (
    pulp.lpSum(annual_fixed_cost[j] * y[j]
               for j in TARGET_WAREHOUSES)
    +
    pulp.lpSum(
        transport_costs[(j, i)] * x[j][i]
        for j in TARGET_WAREHOUSES
        for i in TARGET_FACILITIES
    )
) <= BUDGET_LIMIT

# Solve
prob.solve()

# ============================================================
# 5. RESULTS
# ============================================================

print("\n" + "="*50)
print(f"OPTIMIZATION␣STATUS:␣{pulp.LpStatus[prob.status]}")
print("="*50)

if pulp.LpStatus[prob.status] != 'Optimal':
    print("Solution␣not␣optimal.␣Check␣constraints.")
else:
    total_cost = pulp.value(prob.objective)

    total_fixed = sum(
        annual_fixed_cost[j] * pulp.value(y[j])
        for j in TARGET_WAREHOUSES
    )

    total_transport = sum(
        transport_costs[(j, i)] * pulp.value(x[j][i])
        for j in TARGET_WAREHOUSES
```

```
        for i in TARGET_FACILITIES
    )

    print(f"\nTotal Annual Cost: ${total_cost:,.2f}")
    print(f"  Fixed Cost:      ${total_fixed:,.2f}")
    print(f"  Transport Cost:  ${total_transport:,.2f}")
    print(f"  Remaining Budget: ${BUDGET_LIMIT - total_cost:,.2f}
       ")

    print("\nSelected Warehouses:")
    for j in TARGET_WAREHOUSES:
        if pulp.value(y[j]) > 0.5:
            print(f"  [X] {j}")
        else:
            print(f"  [ ] {j}")

# ============================================================
# 6. VISUALIZATION (NETWORK MAP)
# ============================================================

print("\nGenerating network map...")

fac_coords = facilities_df[
    facilities_df['facility_id'].isin(TARGET_FACILITIES)
].set_index('facility_id')

wh_coords = warehouses_df[
    warehouses_df['warehouse_id'].isin(TARGET_WAREHOUSES)
].set_index('warehouse_id')

plt.figure(figsize=(12, 8))

# ---- Plot Facilities
plt.scatter(fac_coords['longitude'],
            fac_coords['latitude'],
            c='blue',
            s=100,
            label='Facilities',
            zorder=3)

for idx, row in fac_coords.iterrows():
```

```
    plt.text(row['longitude'],
             row['latitude'] + 0.0005,
             idx,
             fontsize=8,
             ha='center')

# ---- Plot Warehouses
for j in TARGET_WAREHOUSES:
    lat = wh_coords.loc[j, 'latitude']
    lon = wh_coords.loc[j, 'longitude']

    if pulp.value(y[j]) > 0.5:
        plt.scatter(lon, lat,
                    c='red',
                    s=250,
                    marker='s',
                    edgecolors='black',
                    zorder=4)
        plt.text(lon,
                 lat - 0.0005,
                 j,
                 fontsize=10,
                 fontweight='bold',
                 ha='center')
    else:
        plt.scatter(lon, lat,
                    c='gray',
                    s=100,
                    marker='x',
                    zorder=2)

# ---- Plot Flow Lines
max_flow = max(
    pulp.value(x[j][i])
    for j in TARGET_WAREHOUSES
    for i in TARGET_FACILITIES
)

for j in TARGET_WAREHOUSES:
    for i in TARGET_FACILITIES:
        val = pulp.value(x[j][i])
```

```
        if val > 0:
            w_lon = wh_coords.loc[j, 'longitude']
            w_lat = wh_coords.loc[j, 'latitude']
            f_lon = fac_coords.loc[i, 'longitude']
            f_lat = fac_coords.loc[i, 'latitude']

            width = 1 + (val / max_flow) * 5

            plt.plot([w_lon, f_lon],
                     [w_lat, f_lat],
                     'k--',
                     linewidth=width,
                     alpha=0.6)

plt.title(
    f'Optimal Distribution Network\nTotal Cost: ${total_cost:,.0f
       }',
    fontsize=14
)

plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()

plt.savefig('optimal_distribution_map.png', dpi=300)
print("Map saved as 'optimal_distribution_map.png'")

plt.show()

# ============================================================
# 7. INTERACTIVE MAP (FOLIUM)
# ============================================================

!pip install folium

import folium
from folium.plugins import MarkerCluster

print("Generating interactive map...")
```

```
# ---- Filter coordinates
fac_coords = facilities_df[
    facilities_df['facility_id'].isin(TARGET_FACILITIES)
].set_index('facility_id')

wh_coords = warehouses_df[
    warehouses_df['warehouse_id'].isin(TARGET_WAREHOUSES)
].set_index('warehouse_id')

# ---- Center map
center_lat = wh_coords["latitude"].mean()
center_lon = wh_coords["longitude"].mean()

m = folium.Map(location=[center_lat, center_lon], zoom_start=14)

# ---- Marker clusters
warehouse_cluster = MarkerCluster(name="Warehouses").add_to(m)
facility_cluster = MarkerCluster(name="Facilities").add_to(m)

# ==========================================================
# Plot Warehouses
# ==========================================================

for wh in TARGET_WAREHOUSES:
    lat = wh_coords.loc[wh, "latitude"]
    lon = wh_coords.loc[wh, "longitude"]

    is_open = pulp.value(y[wh]) > 0.5
    color = "green" if is_open else "red"

    folium.Marker(
        location=[lat, lon],
        popup=(
            f"<b>Warehouse:</b> {wh}<br>"
            f"<b>Open:</b> {'Yes' if is_open else 'No'}<br>"
            f"<b>Annual Capacity:</b> {annual_capacity[wh]:,.0f}"
        ),
        icon=folium.Icon(color=color, icon="industry", prefix="fa
            ")
    ).add_to(warehouse_cluster)
```

```
# ============================================================
# Plot Facilities
# ============================================================

for fac in TARGET_FACILITIES:
    lat = fac_coords.loc[fac, "latitude"]
    lon = fac_coords.loc[fac, "longitude"]

    folium.Marker(
        location=[lat, lon],
        popup=(
            f"<b>Facility:</b>␣{fac}<br>"
            f"<b>Annual␣Demand:</b>␣{annual_demand[fac]:,.0f}"
        ),
        icon=folium.Icon(color="blue", icon="building", prefix="
           fa")
    ).add_to(facility_cluster)

# ============================================================
# Draw Shipment Routes
# ============================================================

max_flow = max(
    pulp.value(x[j][i])
    for j in TARGET_WAREHOUSES
    for i in TARGET_FACILITIES
)

for j in TARGET_WAREHOUSES:
    for i in TARGET_FACILITIES:
        flow = pulp.value(x[j][i])
        if flow > 0:

            w_lat = wh_coords.loc[j, "latitude"]
            w_lon = wh_coords.loc[j, "longitude"]
            f_lat = fac_coords.loc[i, "latitude"]
            f_lon = fac_coords.loc[i, "longitude"]

            # Thickness proportional to shipment
            weight = 2 + (flow / max_flow) * 6
```

```
            folium.PolyLine(
                locations=[[w_lat, w_lon], [f_lat, f_lon]],
                tooltip=f"{j}␣   ␣{i}:␣{flow:,.0f}␣units/year",
                weight=weight,
                color="purple",
                opacity=0.7
            ).add_to(m)

# ---- Layer Control
folium.LayerControl().add_to(m)

# Save map
m.save("interactive_distribution_map.html")
print("Interactive␣map␣saved␣as␣'interactive_distribution_map.
    html'")

# Display map in Colab
m
```